

JOB SHEET 12

Double Linked List

NAMA : M. Javier Thufail

NIM : 254107020019

KELAS : TI 1-G

NO. ABSEN : 18

Percobaan 1 (Operasi Penambahan pada Double Linked List).

Code Mahasiswa18.java:

```
1 package Jobsheet12;
2
3 public class Mahasiswa18 {
4     String nim;
5     String nama;
6     String kelas;
7     double ipk;
8
9     public Mahasiswa18(String nim, String nama, String kelas, double ipk) {
10         this.nim = nim;
11         this.nama = nama;
12         this.kelas = kelas;
13         this.ipk = ipk;
14     }
15
16     public void tampil() {
17         System.out.println(
18             "NIM : " + nim +
19             "\nNama : " + nama +
20             "\nKelas : " + kelas +
21             "\nIPK : " + ipk
22         );
23     }
24 }
```

Code Node18.java:

```
1 package Jobsheet12;
2
3 public class Node18 {
4     Mahasiswa18 data;
5     Node18 prev;
6     Node18 next;
7
8     public Node18(Mahasiswa18 data) {
9         this.data = data;
10        this.prev = null;
11        this.next = null;
12    }
13 }
```

Code DoubleLinkedList18.java:

```
1  package Jobsheet12;
2
3  public class DoubleLinkedList18 {
4      Node18 head;
5      Node18 tail;
6
7      public DoubleLinkedList18() {
8          head = null;
9          tail = null;
10     }
11
12     public boolean isEmpty() {
13         return head == null;
14     }
15
16     public void addFirst(Mahasiswa18 data) {
17         Node18 newNode = new Node18(data);
18
19         if (isEmpty()) {
20             head = tail = newNode;
21         } else {
22             newNode.next = head;
23             head.prev = newNode;
24             head = newNode;
25         }
26     }
27
28     public void addLast(Mahasiswa18 data) {
29         Node18 newNode = new Node18(data);
30
31         if (isEmpty()) {
32             head = tail = newNode;
33         } else {
34             tail.next = newNode;
35             newNode.prev = tail;
36             tail = newNode;
37         }
38     }
39
40     public void insertAfter(String keyNim, Mahasiswa18 data) {
41         Node18 current = head;
42
43         while (current != null && !current.data.nim.equals(keyNim)) {
44             current = current.next;
45         }
46
47         if (current == null) {
48             System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
49             return;
50         }
51
52         Node18 newNode = new Node18(data);
53
54         // Jika current adalah tail, node baru ditambahkan di akhir
55         if (current == tail) {
56             newNode.prev = current;
57             current.next = newNode;
58             tail = newNode;
59         } else { // node baru disisipkan di tengah
60             newNode.prev = current;
61             newNode.next = current.next;
62             current.next.prev = newNode;
63             current.next = newNode;
64         }
65
66         System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
67     }
68
69     public void print() {
70         if (isEmpty()) {
71             System.out.println("Linked List masih kosong.");
72             return;
73         }
74
75         Node18 current = head;
76
77         while (current != null) {
78             current.data.tampil();
79             current = current.next;
80         }
81     }
82 }
```

Code DoubleLinkedListMain18.java:

```
1 package Jobsheet12;
2
3 import java.util.Scanner;
4
5 public class DoubleLinkedListMain18 {
6
7     public static Mahasiswa18 inputMahasiswa(Scanner scan) {
8         System.out.print(s: "Masukkan NIM : ");
9         String nim = scan.nextLine();
10
11         System.out.print(s: "Masukkan Nama : ");
12         String nama = scan.nextLine();
13
14         System.out.print(s: "Masukkan Kelas : ");
15         String kelas = scan.nextLine();
16
17         System.out.print(s: "Masukkan IPK : ");
18         double ipk = scan.nextDouble();
19         scan.nextLine();
20
21         return new Mahasiswa18(nim, nama, kelas, ipk);
22     }
23
24     public static void main(String[] args) {
25         Scanner scan = new Scanner(System.in);
26         DoubleLinkedList18 list = new DoubleLinkedList18();
27
28         int pilihan;
29
30         do {
31             System.out.println(x: "\n===== MENU DOUBLE LINKED LIST =====");
32             System.out.println(x: "1. Tambah data di awal");
33             System.out.println(x: "2. Tambah data di akhir");
34             System.out.println(x: "3. Sisipkan data di tengah");
35             System.out.println(x: "4. Hapus data di awal");
36             System.out.println(x: "5. Hapus data di akhir");
37             System.out.println(x: "6. Tampilkan data");
38             System.out.println(x: "0. Keluar");
39             System.out.print(s: "Pilih menu : ");
40
41             pilihan = scan.nextInt();
42             scan.nextLine();
43
44             switch (pilihan) {
45                 case 1:
46                     Mahasiswa18 mhsAwal = inputMahasiswa(scan);
47                     list.addFirst(mhsAwal);
48                     break;
49                 case 2:
50                     Mahasiswa18 mhsAkhir = inputMahasiswa(scan);
51                     list.addLast(mhsAkhir);
52                     break;
53                 case 3:
54                     System.out.print(s: "Masukkan NIM yang dicari : ");
55                     String keyNim = scan.nextLine();
56                     System.out.println(x: "Masukkan data baru:");
57                     Mahasiswa18 dataBaru = inputMahasiswa(scan);
58                     list.insertAfter(keyNim, dataBaru);
59                     break;
60                 case 4:
61                     list.removeFirst();
62                     break;
63                 case 5:
64                     list.removeLast();
65                     break;
66                 case 6:
67                     list.print();
68                     break;
69                 case 0:
70                     System.out.println(x: "Program selesai.");
71                     break;
72                 default:
73                     System.out.println(x: "Menu tidak valid.");
74             }
75         } while (pilihan != 0);
76
77         scan.close();
78     }
79 }
80 }
```

Outputnya:



Pertanyaan!

1. perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List:
 - Single Linked List hanya punya pointer next jadi, traversal hanya maju. Sedangkan,
 - Double Linked List punya pointer next dan prev jadi, traversal bisa maju dan mundur.
2. next digunakan untuk menuju node berikutnya, sedangkan prev digunakan untuk kembali ke node sebelumnya. Keduanya membantu proses traversal dan manipulasi node.
3. Konstruktor DoubleLinkedList berfungsi mengatur kondisi awal linked list menjadi kosong dengan memberi nilai null pada head dan tail.
4. Karena saat list kosong, node pertama menjadi node awal sekaligus node terakhir, maka head dan tail harus menunjuk node yang sama.
- 5.

```
69     public void print() {
70         if (isEmpty()) {
71             System.out.println(x: "Linked List masih kosong.");
72             return;
73         }
74
75         Node18 current = head;
76
77         while (current != null) {
78             current.data.tampil();
79             current = current.next;
80         }
81     }
82 }
```

6.

```
82     public void printReverse() {
83         if (isEmpty()) {
84             System.out.println(x: "Linked List masih kosong.");
85             return;
86         }
87
88         System.out.println(x: "Menampilkan data secara terbalik (Tail ke Head):");
89         Node18 current = tail;
90         while (current != null) {
91             current.data.tampil();
92             System.out.println(x: "-----");
93             current = current.prev;
94         }
95     }
96 }
97 }
```

You, 10 minutes ago • Percobaan 1 (12.2)

Percobaan 2 (Operasi Penghapusan pada Double Linked List).

Code DoubleLinkedList18.java:

```

Mahasiswa18.java  Node18.java  DoubleLinkedList18.java M  DoubleLinkedListMain18.java
Jobsheet12 > DoubleLinkedList18.java > ...
3 public class DoubleLinkedList18 {
98     public void removeFirst() {
99         if (isEmpty()) {
100             System.out.println(x: "Linked List kosong.");
101             return;
102         }
103
104         if (head == tail) {
105             head = tail = null;
106         } else {
107             head = head.next;
108             head.prev = null;
109         }
110
111         System.out.println(x: "Data awal berhasil dihapus.");
112     }
113
114     public void removeLast() {
115         if (isEmpty()) {
116             System.out.println(x: "Linked List kosong.");
117             return;
118         }
119
120         if (head == tail) {
121             head = tail = null;
122         } else {
123             tail = tail.prev;
124             tail.next = null;
125         }
126
127         System.out.println(x: "Data akhir berhasil dihapus.");
128     }
129 }

```

Outputnya:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\TIVASD\PraktikumAlgoritmaStrukturData> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\TIVASD\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Jobsheet12.DoubleLinkedListMain18'

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 1
Masukkan NIM : 11
Masukkan Nama : Vir
Masukkan Kelas : 1G
Masukkan IPK : 4

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 11
Masukkan data baru:
Masukkan NIM : 22
Masukkan Nama : Ben
Masukkan Kelas : 1G
Masukkan IPK : 3
Data berhasil disisipkan setelah NIM 11

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 4
Data awal berhasil dihapus.

```

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
Masukkan NIM : 33
Masukkan Nama : Did
Masukkan Kelas : 1G
Masukkan IPK : 2

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM : 22
Nama : Ben
Kelas : 1G
IPK : 3.0
NIM : 33
Nama : Did
Kelas : 1G
IPK : 2.0

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 0
Program selesai.
PS C:\TITASD\PraktikumAlgoritmaStrukturData> ^C

```

Pertanyaan!

1. `head = head.next;` berfungsi memindahkan posisi head ke node berikutnya sehingga node pertama lama terhapus dari linked list. Sedangkan `head.prev = null;` berfungsi menghapus hubungan node baru dengan node sebelumnya agar node yang sekarang menjadi head tidak lagi memiliki pointer ke node lama.
- 2.

```

97
98     public void removeFirst() {
99         if (isEmpty()) {
100             System.out.println(x: "Linked List masih kosong, tidak ada yang dihapus.");
101             return;
102         }
103         System.out.println("Menghapus data awal: " + head.data.nama);
104         if (head == tail) {
105             head = tail = null;
106         } else {
107             head = head.next;
108             head.prev = null;
109         }
110     }
111
112     public void removeLast() {
113         if (isEmpty()) {
114             System.out.println(x: "Linked List masih kosong, tidak ada yang dihapus.");
115             return;
116         }
117         System.out.println("Menghapus data akhir: " + tail.data.nama);
118         if (head == tail) {
119             head = tail = null;
120         } else {
121             tail = tail.prev;
122             tail.next = null;
123         }
124     }
125 }

```

You, 9 minutes ago • Uncommitted changes